

RESUMO DO MÓDULO

Comandos SELECT e JOIN - Selecionando dados em mais de uma tabela

Continuando sobre consultas no banco de dados, você vai aprender como realizar consultas envolvendo mais de uma tabela. Para realizar consultas que combinam informações de múltiplas tabelas, precisaremos de um recurso chamado JOIN, que significa "UNIR". O JOIN é uma forma de relacionar as tabelas para cruzar os dados entre elas. Este recurso será demonstrado com exemplos para facilitar o entendimento.

Pesquisando todos os idiomas para saber qual o código do idioma "Português".

```
SELECT * FROM languages;
```

Pesquisando todos os filmes em português

```
SELECT * FROM movies WHERE language_id = 5;
```

15.1 - Usando JOIN para Relacionar Tabelas

Seria mais amigável mostrar a descrição do idioma ao invés do código, certo? Para isso, precisamos utilizar o JOIN. Para o JOIN funcionar, precisamos de um campo que tenha valores em comum entre as tabelas.

No caso das tabelas MOVIES e LANGUAGES, esses campos são o ID na tabela LANGUAGE e o LANGUAGE_ID na tabela MOVIES.

Exemplo de JOIN:

```
SELECT title, release_date, name
```

```
FROM movies
```

```
JOIN languages ON languages.id = language_id
```

```
WHERE language_id = 5;
```

15.2 - Usando Alias para Simplificar Consultas

Às vezes, os nomes das tabelas são um pouco extensos e, para evitar digitar muitos caracteres repetidamente, podemos dar um apelido para a tabela, chamado alias. Para definir um alias, basta colocar o nome que queremos ao lado do nome da tabela.

Exemplo de alias:

```
SELECT title, release_date, name  
FROM movies  
JOIN languages lang ON lang.id = language_id  
WHERE language_id = 5;
```

15.3 - Relacionando Mais de Duas Tabelas

Para exibir a descrição do gênero, precisamos fazer a mesma operação de JOIN que fizemos para o idioma.

Exemplo de JOIN com múltiplas tabelas:

```
SELECT title, release, lang.name, genres.name  
FROM movies  
JOIN languages lang ON lang.id = language_id  
JOIN genres ON genres.id = genre_id  
WHERE language_id = 5;
```

15.4 - Evitando Ambiguidades com Alias

O campo name aparece em ambas as tabelas genres e languages, e o SQL não sabe qual queremos utilizar se não informarmos o nome da tabela de origem ou um apelido. Por isso, você

também deve ter notado que temos dois campos chamados name na nossa consulta.

Podemos aplicar o conceito de alias nos nomes dos campos para deixar nossa consulta mais amigável.

Exemplo:

```
SELECT title, release_date, lang.name AS language_name, genres.name AS genre_name
FROM movies
JOIN languages lang ON lang.id = language_id
JOIN genres ON genres.id = genre_id
WHERE language_id = 1;
```

15.5 - Boas Práticas para Relacionamentos entre Tabelas

Quando estamos fazendo uma consulta que envolve mais de uma tabela, é importante que os campos que usamos para ligar uma tabela a outra sejam chaves primárias em suas tabelas de origem. Os campos de referência na tabela principal (aquela descrita depois do FROM) devem ser chaves estrangeiras. Quando definimos esses campos corretamente, o banco de dados realiza uma busca otimizada para exibir os resultados da forma mais rápida possível.

É essencial sempre considerar esses pontos ao construir consultas envolvendo múltiplas tabelas para garantir desempenho e consistência dos resultados.
